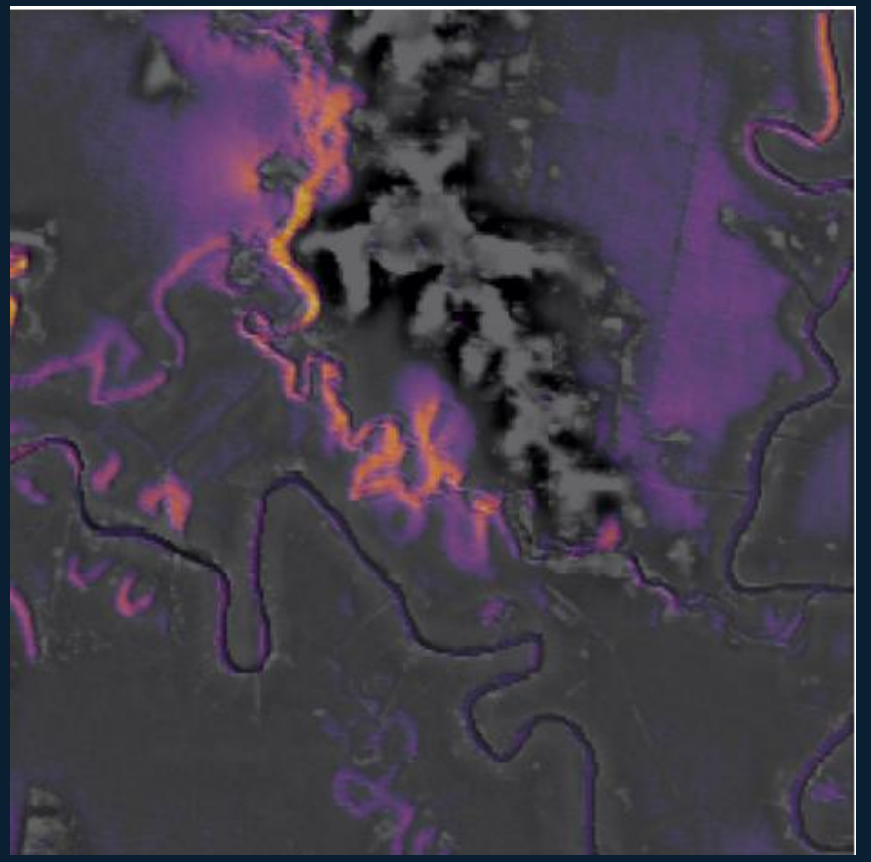
Deep Learning · CA3 Project

# CNN-LSTM Based Spatiotemporal Flood Evolution Forecasting

*Does a neural network forecast floods better than very simple rules?*

Anhad Mahajan · PRN 24070126031
Aman Sumeet Saxena · PRN 24070126030
AIML A2 · October 2026



A picture from our model on the Australia flood. The bright lines are the river channels it pays most attention to.

The problem

# Why we want to forecast floods with a neural network

## Floods are fast

Floods damage homes, crops and roads every year. People can protect much more if they are warned early.

## Simulators are slow

A physics simulator draws very detailed flood maps, but one large flood can take days to simulate on a powerful GPU.

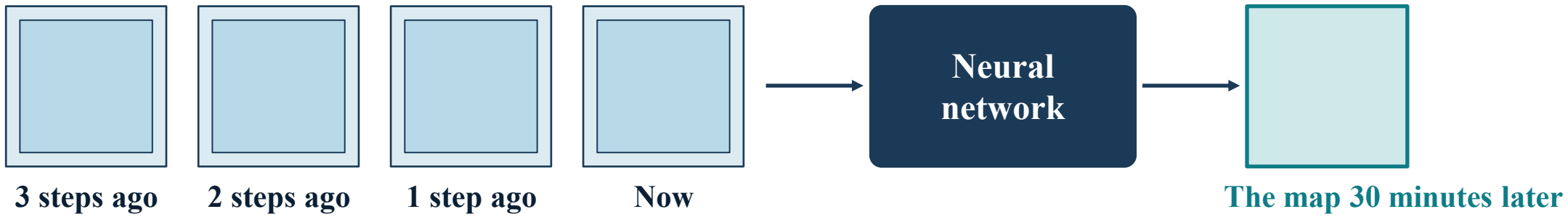
## A network is fast

Once it is trained, a neural network gives a forecast in seconds, so it could take the place of the simulator.

**So our question is simple: is the neural network really better than very simple forecasts?**

The task

# What we ask the model to do




The last four depth maps, 30 minutes apart, plus the rain that has already fallen and the height of the ground.

## The catch

In 30 minutes the water in most places hardly changes. So just copying the last map is already a very good forecast. A neural network is only useful if it does better than simple forecasts like this one.

Part 1 (Colab notebook): one map, 30 minutes ahead. Part 2 (laptop study): five maps in a row, up to 2.5 hours ahead.

# The data: four floods made by a simulator

## Australia

20 Feb to 2 Mar 2022
1073 × 1073 squares
each square 30 m wide
10 days, 481 maps

**Used for training**

## UK

4 to 7 Dec 2015
170 × 275 squares
each square 30 m wide
3 days, 145 maps

**Kept for testing**

## Pakistan

18 to 31 Aug 2022
810 × 441 squares
each square 480 m wide
14 days, 673 maps

**Used for training**

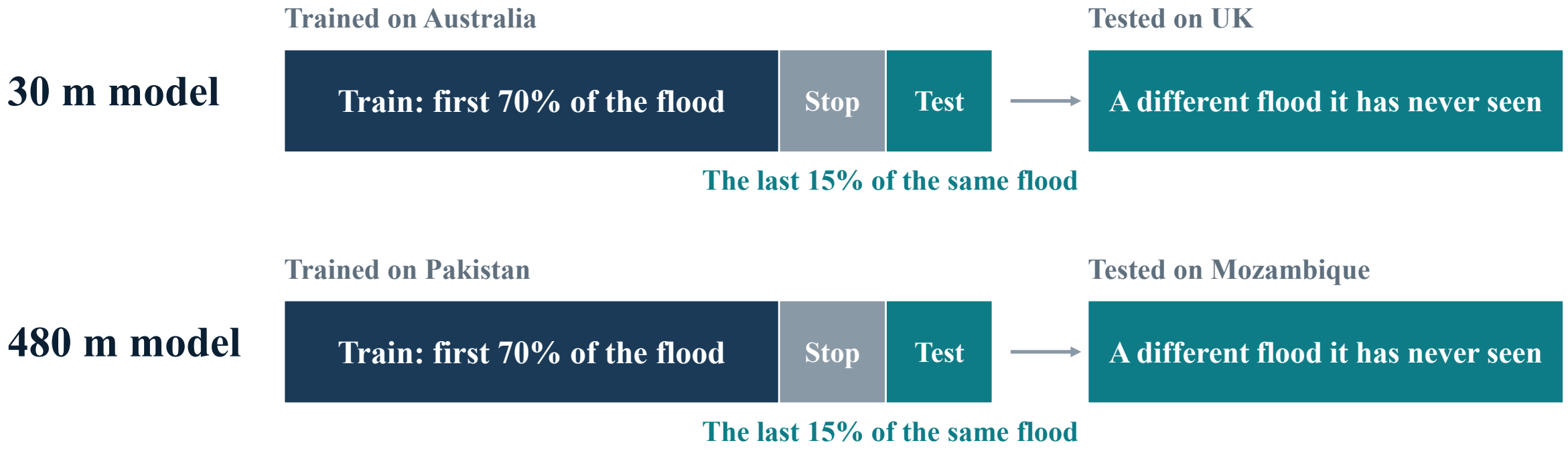
## Mozambique

14 to 20 Mar 2019
151 × 138 squares
each square 480 m wide
6 days, 289 maps

**Kept for testing**

These maps are not photographs. A physics simulator made them from real ground heights, a real land-cover map and real satellite rain. The dataset is called FloodCastBench (Xu et al., 2025).

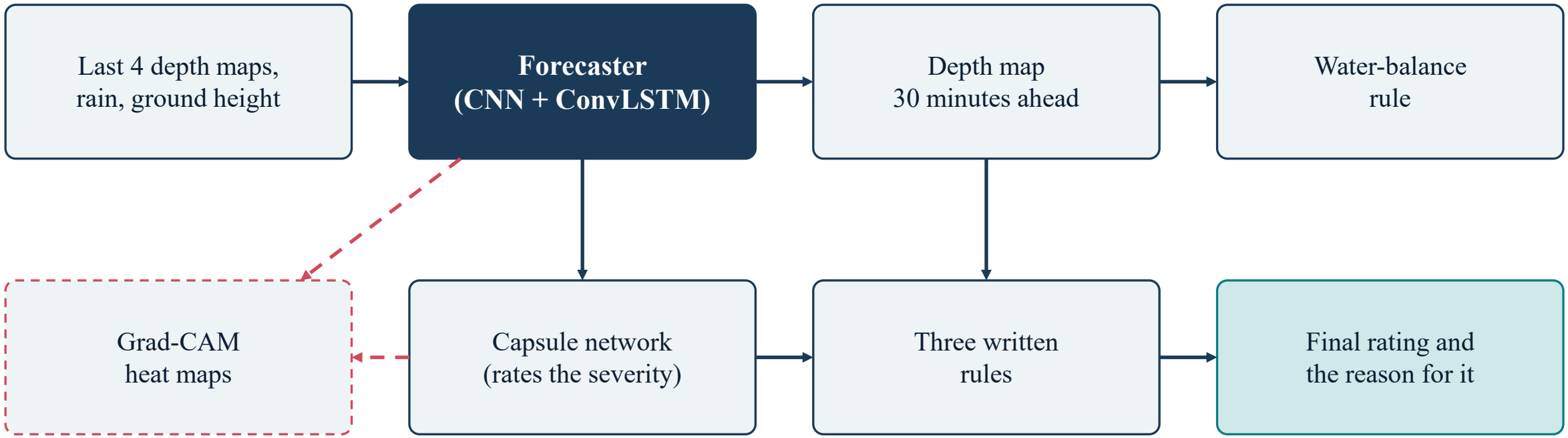
# How we test the model




We test every model twice: on the last part of the flood it learned from, and on a whole new flood. The “Stop” part (15%) is only used to decide when to stop training. We use a separate model for each map size, because a 30 m square and a 480 m square behave very differently.

The system

# Our whole system on one page




The forecaster predicts the next map, and the water-balance rule checks the total water against the rain. The capsule network rates how serious the flood is, and three written rules check that rating.

**NeuroSymbolic AI:** a neural network checked by rules people can read. The written rules are our NeuroSymbolic part.

**Explainable AI:** showing why a model gave its answer. The dashed arrows are Grad-CAM, our way of doing that.

The model

# The forecaster: how it reads the maps




- The CNN (convolutional neural network) layers look at each map on its own and pick out useful shapes, such as water edges, slopes and patches of rain.
- The ConvLSTM layer then reads the four maps in order and keeps a memory of how the water has been moving.
- The model predicts how much the depth will change, not the whole map. This is easier, because most of the map hardly changes in 30 minutes.

**62,833**

parameters, the numbers the model learns while it trains

# How we score a forecast

**skill = 1 − (error of the forecast ÷ error of copying the last map)**




## Copy the last map

The forecast is simply the last map we have, so its skill is 0 by definition.

## The trend forecast

Take the last map and add a fraction of the change seen in the last 30 minutes. It uses one number found from the training data, and no neural network.

# Two versions of the model

## Model A

starts from the last map

next depth = last depth + the change the model predicts
(and never below zero)

This was our first version. It beat copying the last map in three of the four tests, so it looked successful. But in the last part of the Australia flood its skill was −1.449, which is worse than copying.

## Model B

starts from the trend forecast

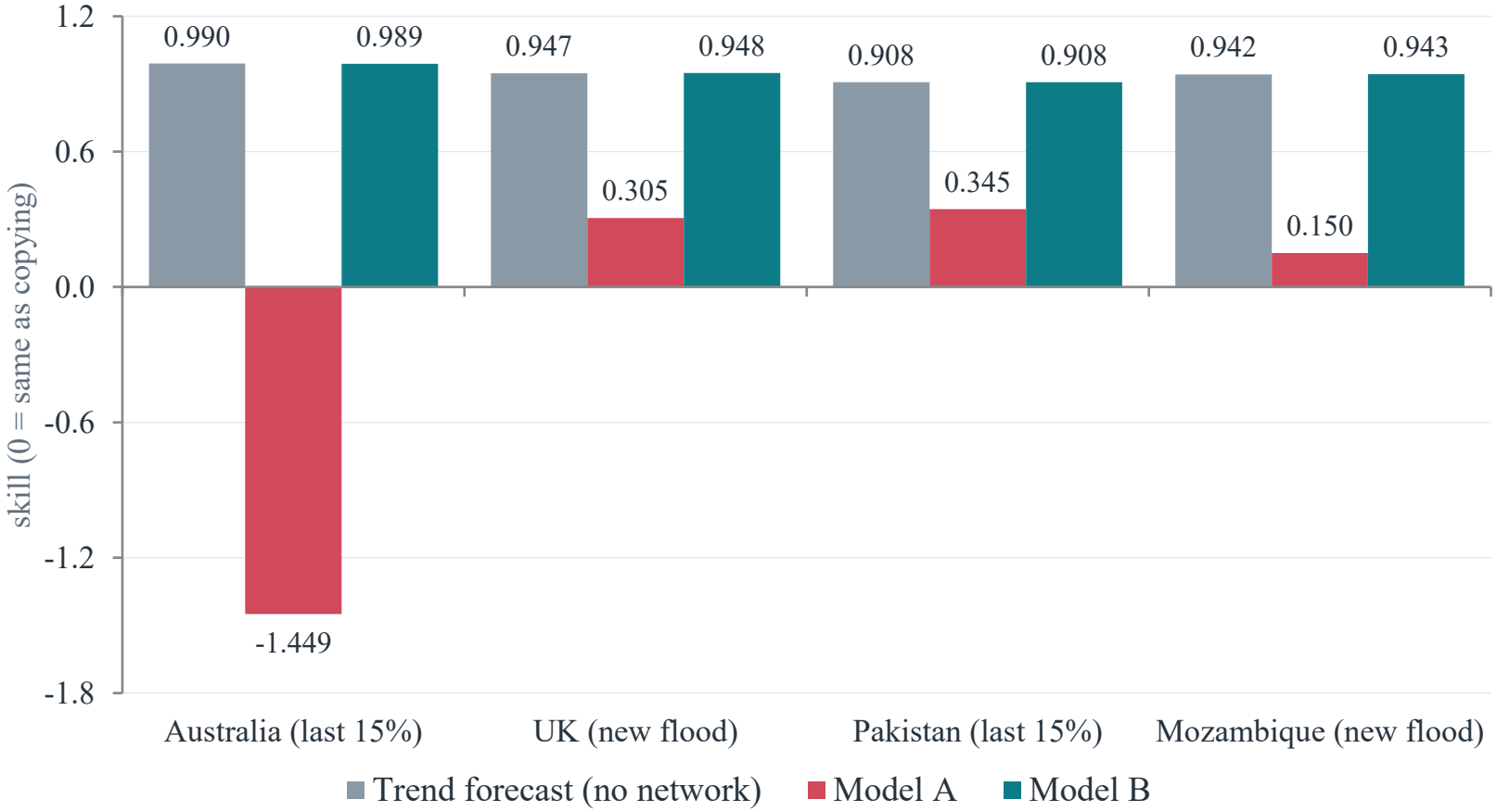
trend = last map + β × (last map − the map before it)
next depth = trend + a small correction from the model

The network only has to learn a small correction. β is one number found from the training data: 0.921 for the 30 m floods and 0.948 for the 480 m floods.

In the Colab notebook, Model A is called the first version and Model B is called version 2.

Part 1: the Colab notebook, forecasting 30 minutes ahead

# The simple trend forecast beats our first model




**−1.449**

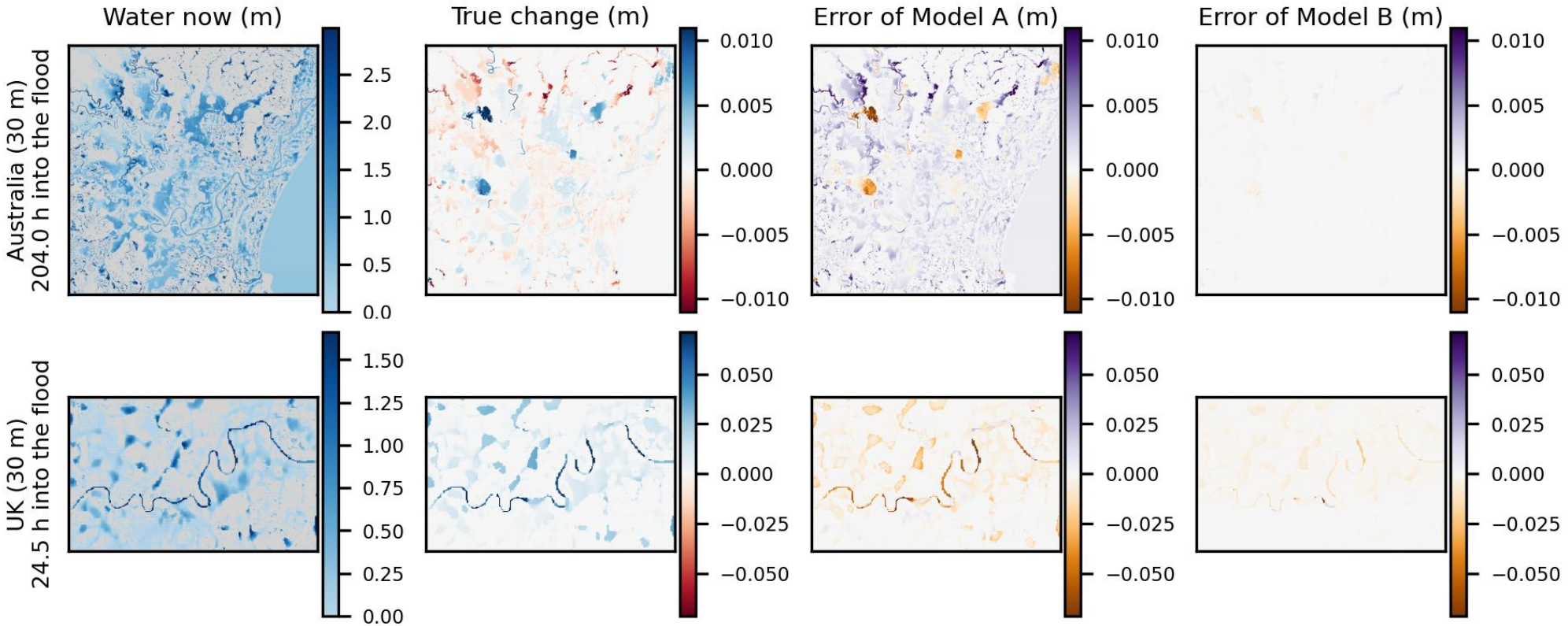
This is Model A in the last part of the Australia flood. It is worse than just copying the last map.

**0.001**

Model B stays within this of the trend forecast in every test. It matches the trend, but it does not beat it.

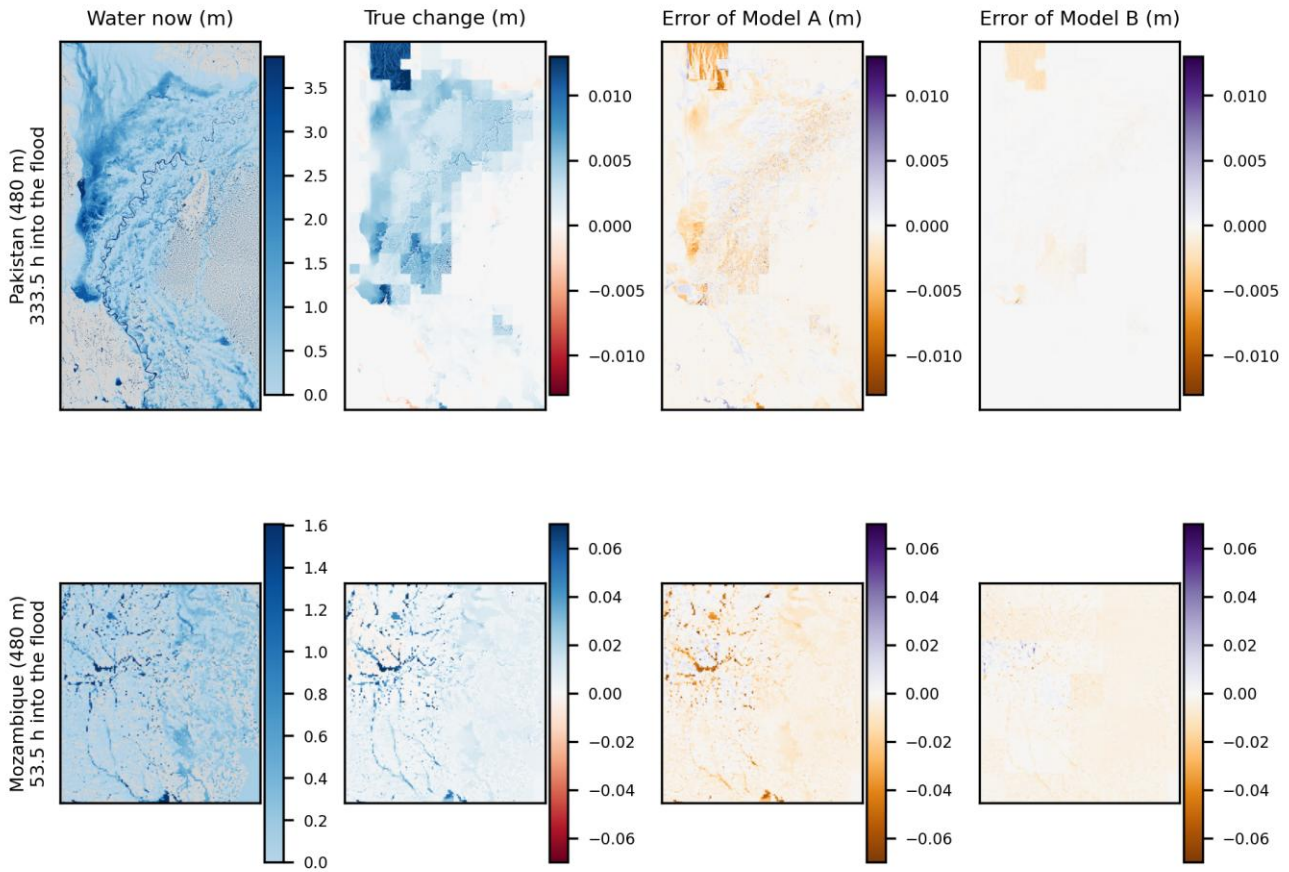
Higher is better. “Last 15%” is the end of the flood the model learned from. “New flood” was never seen in training. All results are single runs.

# On the map, Model B's mistakes almost disappear




Australia (top) and the UK (bottom). Purple means too much water was forecast, and orange means too little. The three maps on the right use the same colour scale.

# The same maps for the two 480 m floods




Pakistan (top) and Mozambique (bottom). Purple means too much water was forecast, and orange means too little. Model B's map is almost blank again.

# At 30 minutes there is very little left to improve

| 1% | 5% | 9% | 6% |
|---|---|---|---|
| **Australia** | **UK** | **Pakistan** | **Mozambique** |

This is how much of the error of copying the last map is still left, after the trend forecast has done its job. Only this much is left for any model to remove.

So Model B matches the trend because there is almost nothing left to add. A fair test of the LSTM's memory needs a longer forecast, 6 to 24 hours ahead, where a simple trend should get worse. That is the next step we want to take.

# Part 2: a bigger study on a laptop

**310** training runs in total

**3** experiments: Australia to UK at 60 m (E60) and 30 m (E30), and Pakistan to Mozambique at 480 m (E480)

**3** splits of the data (folds) and 3 rain settings: past rain only, the original input, and future rain as well

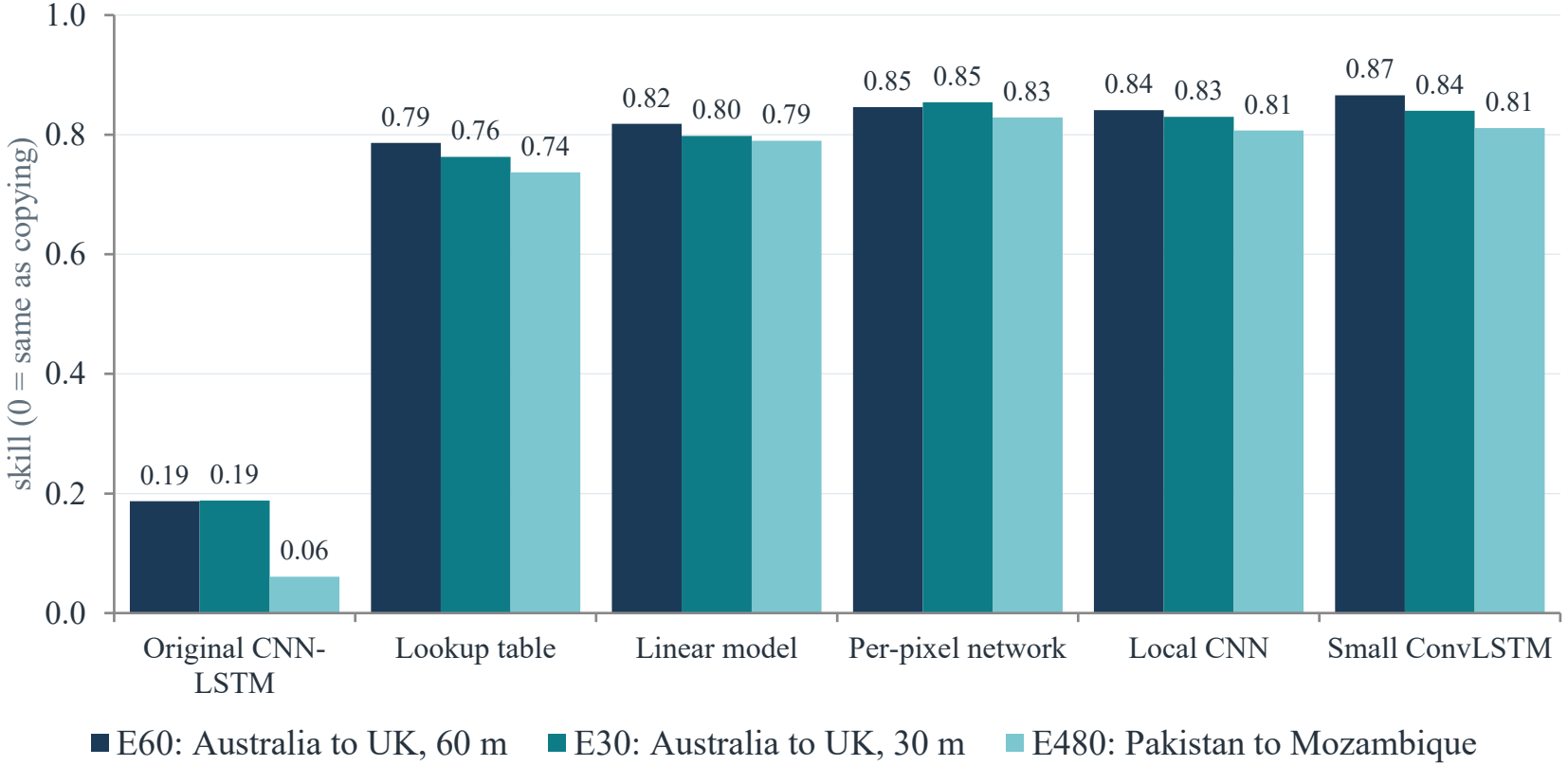
**6** predictions (hypotheses) written down before the final runs

## Models we compared

| | Number of parameters |
|---|---|
| **Original CNN-LSTM** | about 4.5 million |
| **Local CNN** | 4,101 |
| **Small ConvLSTM** | 4,741 |
| **Heavy ConvLSTM** | 652,918 |
| Simple baselines: linear model, lookup table, per-pixel network | very few |

Part 1 and Part 2 use different set-ups, so we do not compare their numbers with each other.

# A very simple linear model already gets most of the way




**0.79 to 0.82**

is the skill of a very simple linear model (a straight-line formula) that only uses the recent depths of each square.

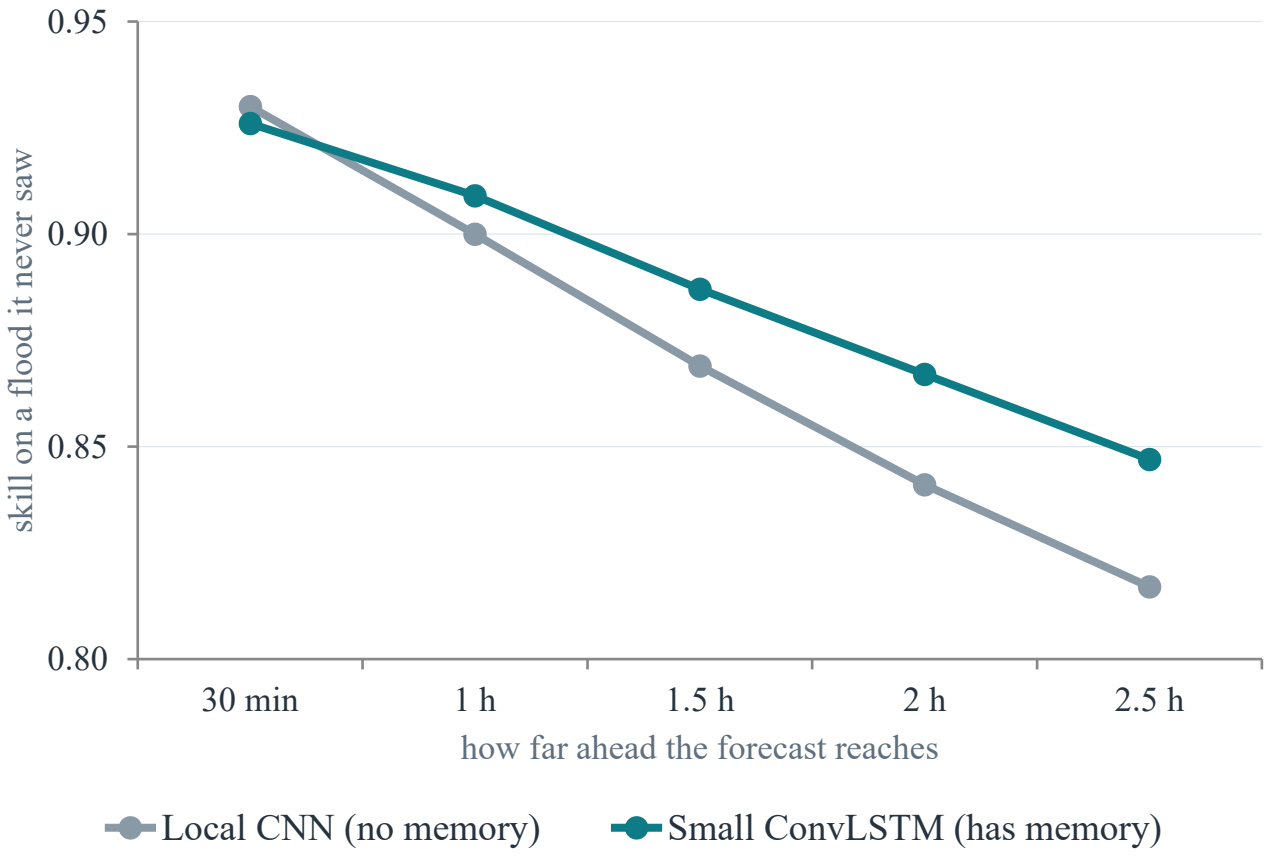
**+0.01 to +0.05**

is all that the local CNN and the small ConvLSTM add on top of it.

Lookup table: a table that only uses the current depth and the last change of each square. Per-pixel network: a small network that works on one square at a time. The original CNN-LSTM scored lowest because it ignores how the depth has been changing.

Part 2: the laptop study

# Does the LSTM's memory help?



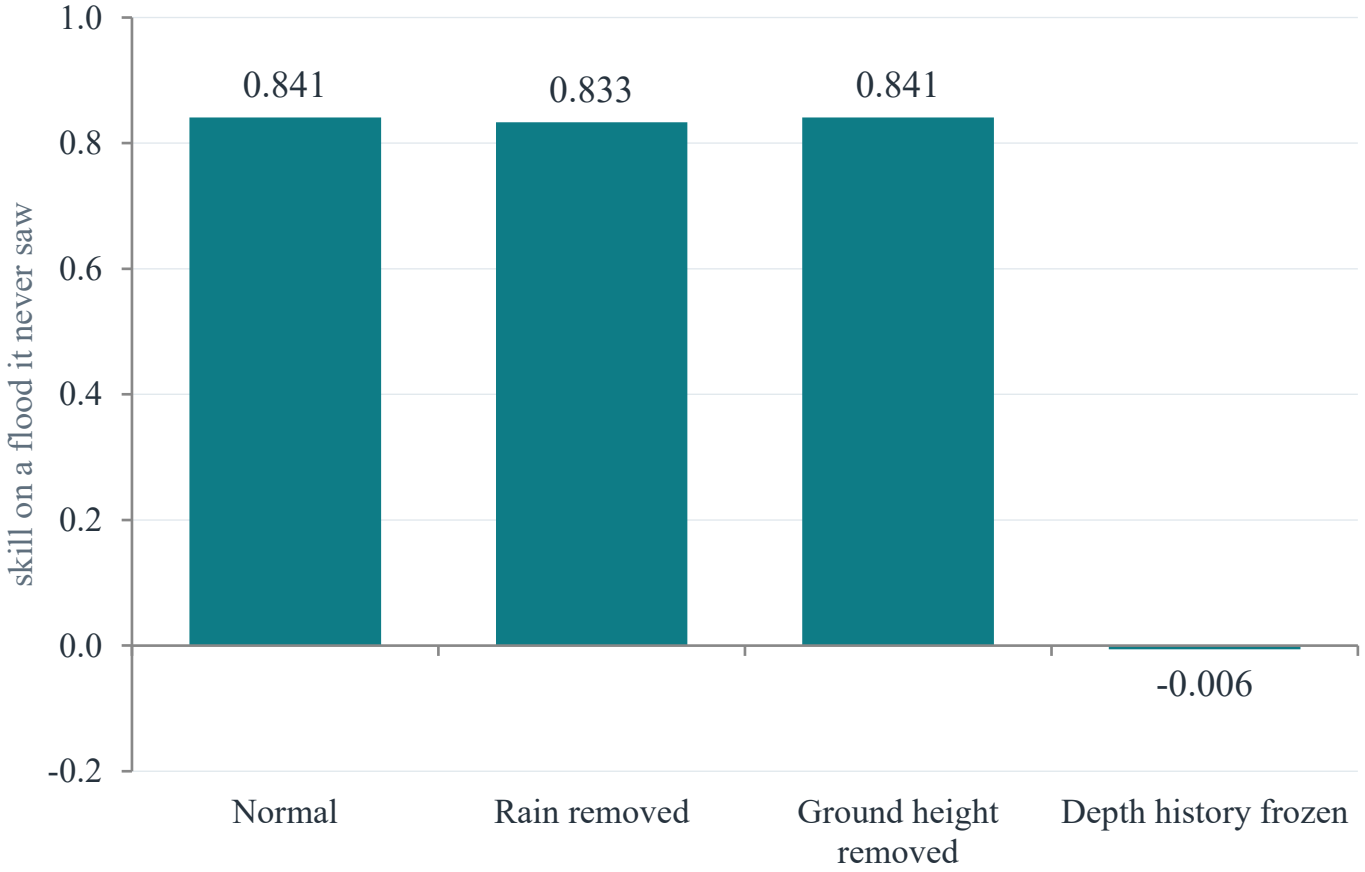

When both models are trained for the same length (90 passes through the training data), they score the same. The difference is +0.004 on the UK and −0.014 on Mozambique, inside the ±0.03 band we set in advance to mean “the same”. A model 140 times bigger did not do better either.

The ConvLSTM does lose less skill as the forecast reaches further ahead. At 30 minutes the plain CNN is slightly better, and at 2.5 hours the ConvLSTM is better. It also stayed stable when we hid the start of a flood from training.

We did not test whether the memory is the reason for these two differences.

The chart shows Australia to UK at 60 m, with the planned 30-pass training. The other two experiments show the same pattern.

# The models mostly use the recent depths




## ≈ 0

If we freeze the depth history, so the model cannot see any change over time, its skill drops to about zero.

## 0.01 to 0.02

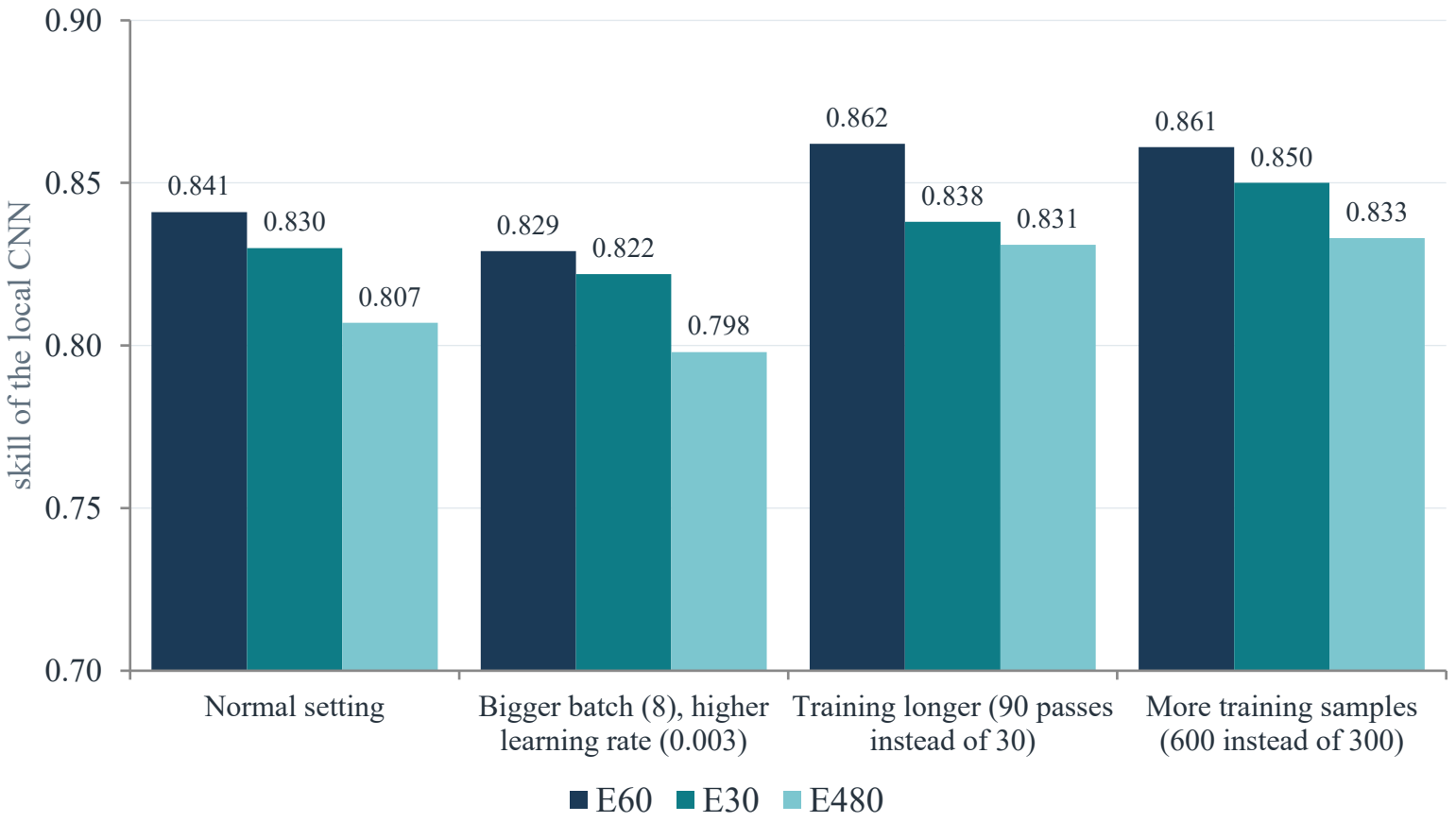
Taking the rain away costs only about this much. Taking the ground height away costs 0.006 or less.

## 0.84 → 0.17

When we train the local CNN without the depth history, its skill falls from 0.84 to 0.17.

The bars show the local CNN on the UK flood (60 m). Only our original CNN-LSTM relied on the rain instead of the depth history.

# What we changed while tuning, and what helped




**−0.008 to −0.026**

A bigger batch with a higher learning rate made the score a little lower.

**+0.008 to +0.024**

Training for longer helped a little.

**0.006 to 0.010**

This is how much the score moves just from changing the random seed. So differences below about 0.02 are too small to trust.

E60, E30 and E480 are the three experiments. We only tested the Adam optimiser and a few learning rates, and we did not test other optimisers.

Part 2: the laptop study

# Six predictions we wrote down before the final runs

**1** The local CNN and the small ConvLSTM beat copying the last map

**Held in 13 of 16 test cases**

**2** They also beat the best linear model

**Held in 15 of 16 cases, but only 8 by a clear margin**

**3** The recent depth history matters

**Held in all three experiments**

**4** The LSTM's memory matters

**Mixed: the same score at equal training**

**5** More rain information helps

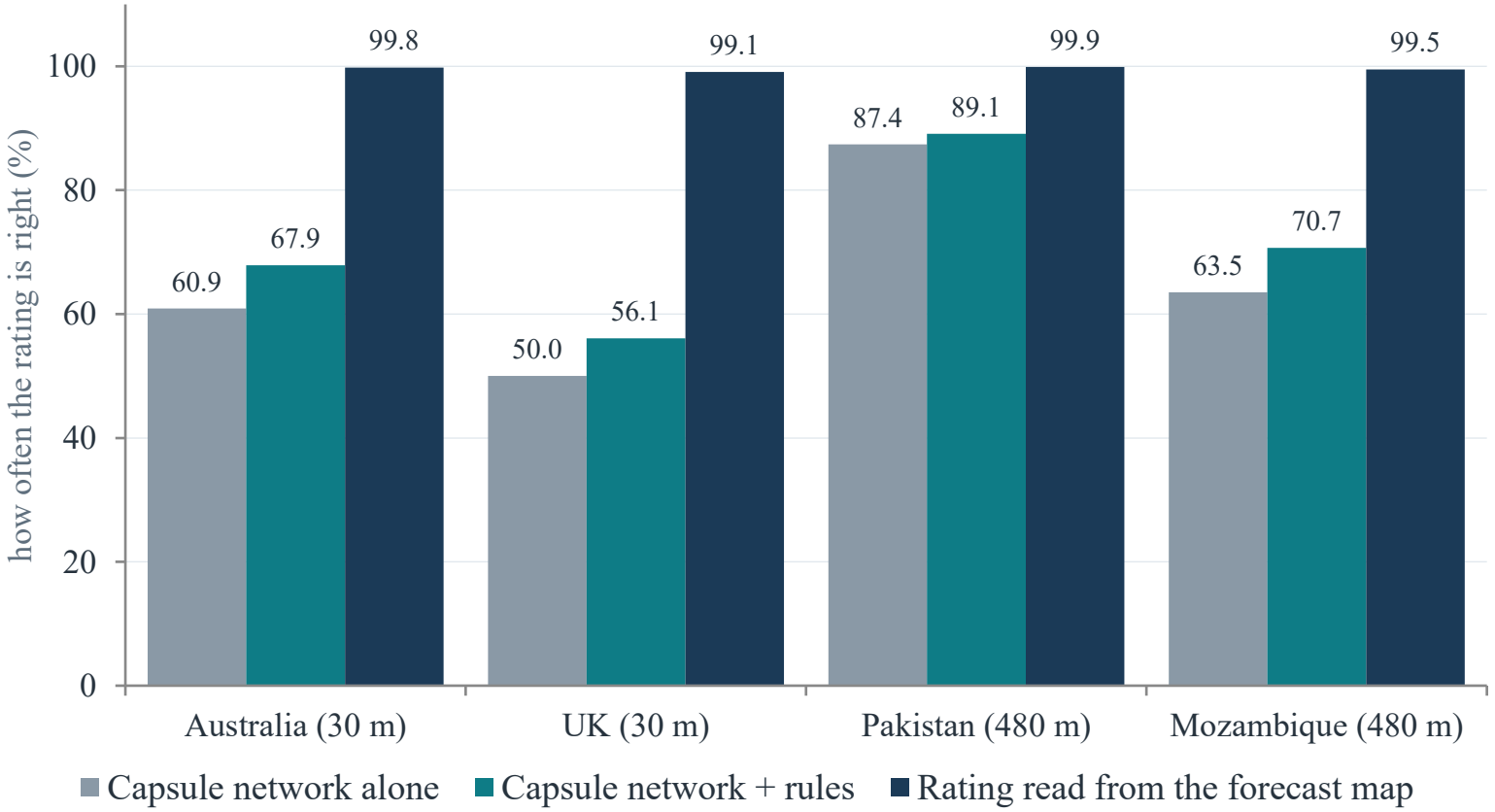
**Held for the local CNN, not the ConvLSTM**

**6** A stricter test gives a similar score

**Held in only 2 of 5 cases**

Green means it held. Orange means it held only in part, or the answer was mixed. Red means it held in only 2 of 5 cases.

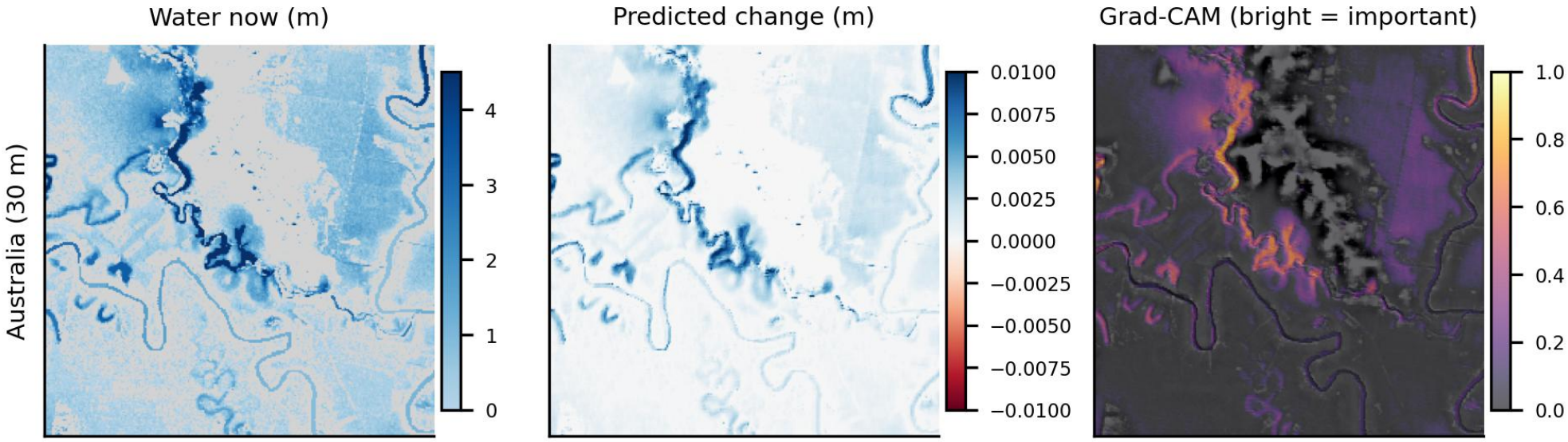
# Rating how serious the flood is




- The capsule network beat a plain output layer on Australia at first (67.8% against 60.2%). With rotated and flipped copies of the data added, they were almost the same (80.3% against 79.8%).
- Three written rules check the rating and always give a reason. A neural network checked by rules people can read is called NeuroSymbolic AI. The rules raised the accuracy in all four floods.
- In Pakistan, 698 of the 1,000 test tiles are “severe”. So always answering “severe” would already be right 69.8% of the time.

Tiles are small pieces of the map, 1,000 per flood. The rating read from the forecast map is right about 99% of the time, because the rating 30 minutes ahead is almost the same as the rating now.

# Where does the forecaster look?




**Explainable AI** means showing why a model gave its answer. Grad-CAM is our way of doing that: it colours the parts of the input that mattered most, and bright means more important.

## For the forecaster

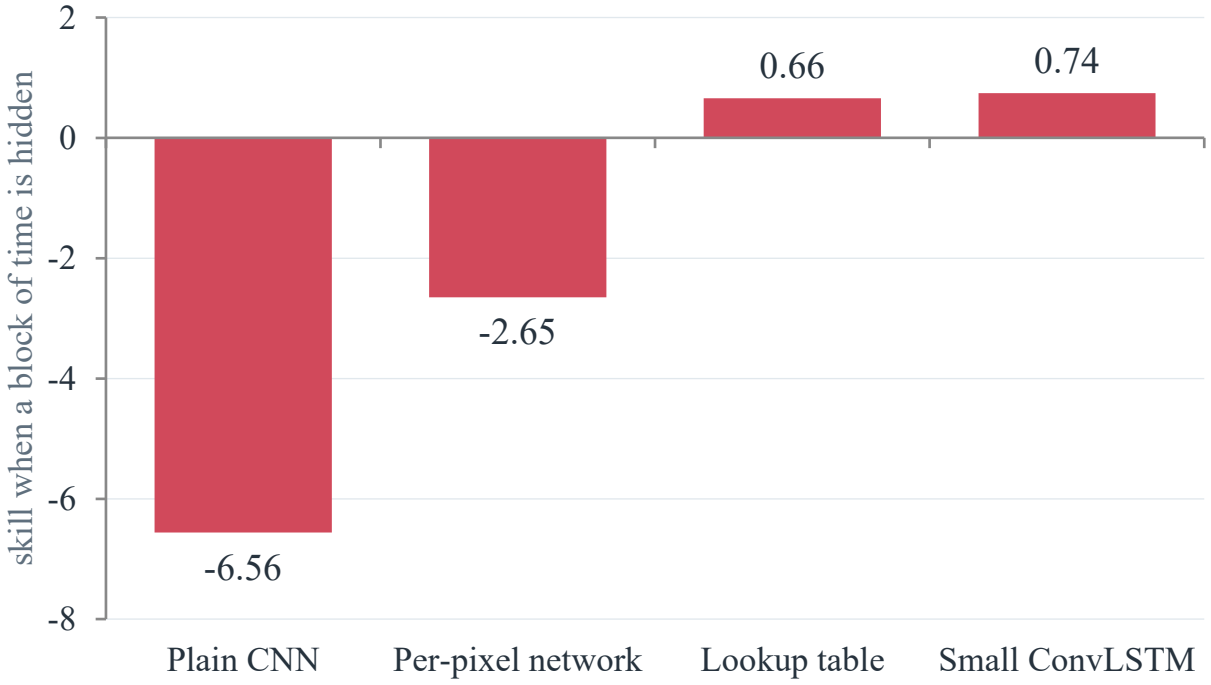
The bright areas follow the river channels and flood edges, where the water is changing. When we deleted the top 10% of the heat map, the forecast changed more than when we deleted random cells, in 84% to 92% of the test samples.

## For the capsule network

The heat maps were weak. Maps from two training runs agreed only a little (rank correlation 0.17). So we show them as pictures, not as proof.

Where the model fails, and the limits of our study

# What this study cannot tell us




In a harder test we hid a block of time from training. The plain CNN then failed badly on Pakistan, because six of 480 test windows, all at the very start of the flood, caused 98% of its error. The ConvLSTM and the lookup table coped.

## The limits of this study

- We have only one flood per region, so there are only two truly independent train-and-test pairs.
- The floods are simulated, so the models learn to copy a simulator, not real floods.
- The Colab results come from single runs.
- The capsule network and Grad-CAM were built on Model A, not on Model B.

Live demo

# The Colab notebook

1. Load the four floods and cut them into small training pieces
2. Load Model A and test it on whole maps
3. Fit the trend forecast and load Model B
4. Run the capsule network, Grad-CAM and the written rules
5. Print one summary table

## Nothing is retrained

Colab is Google's online notebook, and it gives us a GPU. The notebook loads the models we already saved on Google Drive. We ran it, and every number matched our report.

It takes about 20 to 30 minutes to run.

Conclusion

# What we learned

1. At 30 minutes ahead, a simple trend forecast with one fitted number does most of the work. Our second model matches it, but it does not beat it.

2. Up to 2.5 hours ahead, a very simple linear model of the recent depths already scores 0.79 to 0.82. The best networks score 0.81 to 0.87, and the LSTM's memory did not help when the training length was equal.

3. Always compare a model with simple forecasts. Our first model looked good until we did.

**What we want to do next:** forecast 6 to 24 hours ahead, where a simple trend should get worse and the LSTM's memory could really matter. We also want to rebuild the capsule network and Grad-CAM on Model B, and repeat the Colab runs with several random seeds.

The main works we built on

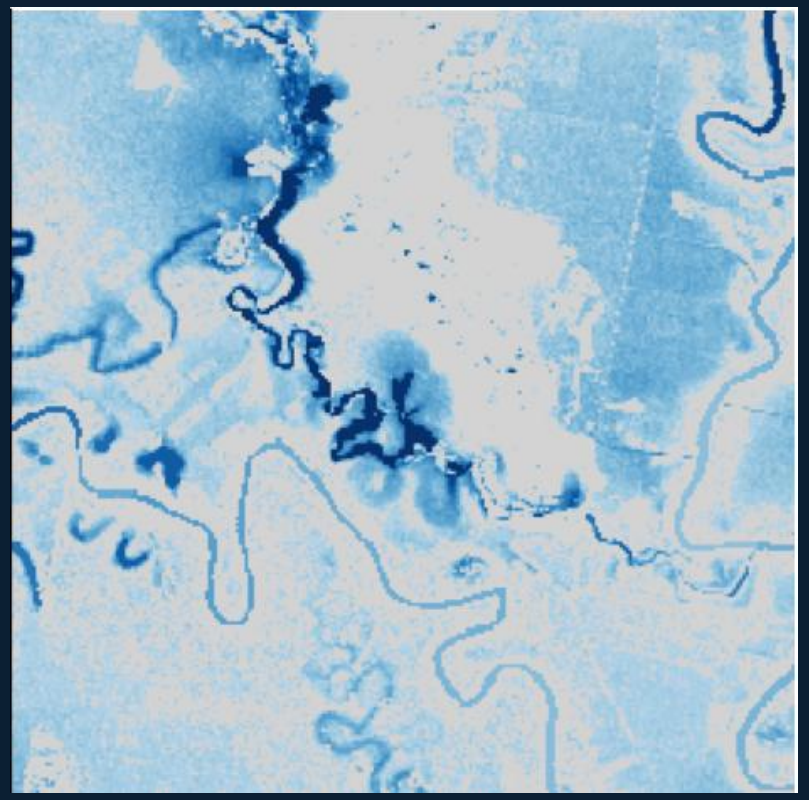
# References


- Q. Xu, Y. Shi, J. Zhao and X. X. Zhu, FloodCastBench: a large-scale dataset and foundation models for flood modeling and forecasting, Scientific Data, 2025.
- X. Shi et al., Convolutional LSTM network: a machine learning approach for precipitation nowcasting, NIPS 2015.
- S. Sabour, N. Frosst and G. E. Hinton, Dynamic routing between capsules, NIPS 2017.
- R. R. Selvaraju et al., Grad-CAM: visual explanations from deep networks via gradient-based localization, ICCV 2017.
- S. Makridakis, E. Spiliotis and V. Assimakopoulos, Statistical and machine learning forecasting methods: concerns and ways forward, PLOS ONE, 2018.
- A. Zeng, M. Chen, L. Zhang and Q. Xu, Are transformers effective for time series forecasting? AAAI 2023.
- D. P. Kingma and J. Ba, Adam: a method for stochastic optimization, ICLR 2015.


The full list of 22 references, with DOIs, is in the report.

# Thank you

Anhad Mahajan · PRN 24070126031
Aman Sumeet Saxena · PRN 24070126030
AIML A2



The water depth in the Australia flood, one step before the forecast.